



Mateo C [Redacted]

[Grupo: [Redacted]]

Docente: [Redacted]

Analista en Tecnologías de la Información

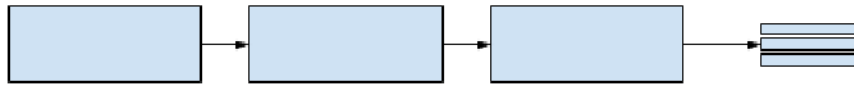
Fecha de entrega del documento (19-11-2025)

Índice

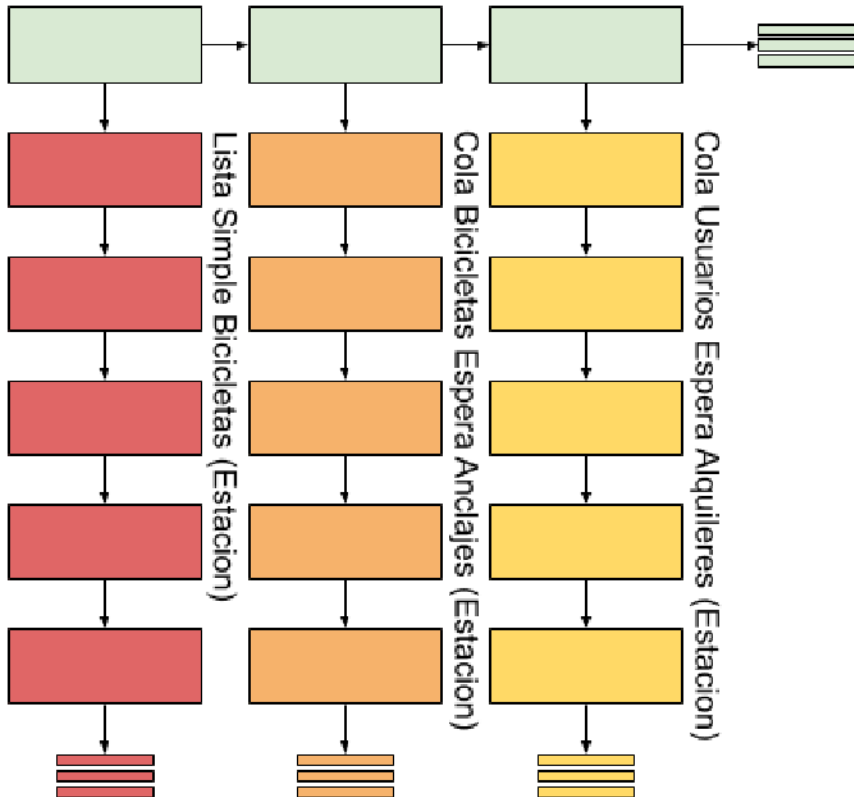
1. Diagrama Ilustrativo de Estructuras.....	3
2. Clases Adicionales Implementadas.....	4
2.1 Usuario.....	4
2.2 Bicicleta.....	4
2.3 BicicletaEstado.....	4
2.4 Deposito.....	4
2.5 Estacion.....	4
2.6 Validadores.UsuarioV.....	5
2.7 Validadores.BicicletaV.....	5
2.8 Comparadores.UsuarioCmp.....	5
2.9 Comparadores.EstacionCmp.....	5
3. Justificación sobre Decisiones de Diseño.....	5
3.1 ListaSE<Bicicleta> en Deposito.....	5
3.2 ListaSE<Usuario> en Sistema.....	5
3.3 ListaSE<Estacion> en Sistema.....	6
3.4 PilaSE<String> para Historial de Alquileres en Sistema.....	6
3.5 ListaSE<Bicicleta> en Estacion.....	6
3.5 ColaSE<Bicicleta> y ColaSE<Usuario> en Estacion.....	6

1. Diagrama Ilustrativo de Estructuras

Lista Simple Usuarios (Sistema) Ordenada Alfabéticamente



Lista Simple Estaciones (Sistema) Ordenada Alfabéticamente



Pila Historial Alquileres (Sistema)



Lista Doble Bicicletas (Deposito)



2. Clases Adicionales Implementadas

2.1 Usuario	
m_cedula	String
m_nombre	String
m_bicicletaActual	Bicicleta
m_cantidadAlquileres	int

2.2 Bicicleta	
m_codigo	String
m_tipo	String
m_estado	BicicletaEstado
m_cantidadAlquileres	int

2.3 BicicletaEstado	
Disponible	0
Mantenimiento	1
Alquilada	2

2.4 Deposito	
m_bicicletas	ListaSE<Bicicleta>

2.5 Estacion	
m_nombre	String
m_barrio	String
m_capacidad	int
m_bicicletas	ListaSE<Bicicleta>

m_colaEsperaAnclajes	ColaSE<Bicicleta>
m_colaEsperaUsuarios	ColaSE<Usuario>

2.6 Validadores.UsuarioV	
CANTIDAD_DIGITOS	int

2.7 Validadores.BicicletaV	
CODIGO_LONGITUD	int
TIPOS_VALIDOS	String[]

2.8 Comparadores.UsuarioCmp	
porNombreAscendente	Comparator<Usuario>

2.9 Comparadores.EstacionCmp	
porNombreAscendente	Comparator<Estacion>

3. Justificación sobre Decisiones de Diseño

3.1 ListaSE<Bicicleta> en Deposito

El requerimiento 3.3 Listar bicis en depósito solicita hacerlo tal que: “en el orden en el cual fueron ingresadas”; una Lista Simplemente Enlazada es una colección lineal, lo que significa que cada inserción agrega el elemento al final de sí misma, logrando el orden solicitado por defecto.

3.2 ListaSE<Usuario> en Sistema

El requerimiento 3.2 Listar usuarios solicita hacerlo tal que: “ordenados creciente por nombre”. Una Lista Simplemente Enlazada es una colección lineal que permite recorrer sus elementos de forma secuencial, siendo apropiada cuando no se requiere acceso aleatorio.

La interfaz `ILista` dispone del método `adicionarOrdenado`, el cual recibe un **criterio de ordenación** (en este caso, un `Comparator<Usuario>` que ordene alfabéticamente por la propiedad `nombre`).

Al insertar un nuevo elemento, la lista (que ya se encuentra ordenada según este mismo criterio, pues se habrán hecho inserciones anteriores siguiendo el mismo criterio) se recorre **secuencialmente** hasta encontrar el primer nodo cuyo valor sea “mayor” que el nuevo elemento. En ese punto, se **ajustan los punteros `siguiente`** de ambos nodos:

- El puntero del nodo actual pasa a apuntar al nuevo elemento.
- El puntero del nuevo elemento pasa a apuntar al nodo que le resultó mayor.

De esta manera, la inserción mantiene el orden alfabético de la colección **sin necesidad de reordenar la lista completa**, garantizando la eficiencia y simplicidad que el problema requiere.

3.3 ListaSE<Estacion> en Sistema

No se encontró una justificación real sobre la selección de ésta estructura en relación a los requerimientos, pero una Lista Simple siempre será mucho más liviana que una Lista Doble, si no hay razón para emplear una Lista Doble, pero sabemos que el número de estaciones será dinámico aún así, una Lista Simple debería ser más que suficiente.

3.4 PilaSE<String> para Historial de Alquileres en Sistema

El requerimiento 2.11 Deshacer últimos retiros describe el caso de uso ideal para ésta estructura, ya que el comportamiento descrito es un **LIFO** clásico. La última acción realizada (último retiro) debe revertirse primero. `PilaSE` es la estructura óptima para soportar la reversión secuencial de las últimas operaciones, manteniendo el orden inverso natural de las acciones.

3.5 ListaSE<Bicicleta> en Estacion

Se emplea una ListaSE para representar la realidad de una Estacion, en la cual las bicicletas son colocadas en orden de llegada (secuencial).

3.5 ColaSE<Bicicleta> y ColaSE<Usuario> en Estacion

Los requerimientos 2.9 Alquilar Bicicleta y 2.10 Devolver Bicicleta describen claramente un comportamiento **FIFO**:

1. El primer usuario que queda en espera es el primero en recibir una bici disponible.
2. Si no hay espacio en la estación, la bicicleta devuelta espera su turno para anclarse.